Listing 4.2 Essential parts of an async test

```
import akka.actor.testkit.typed.scaladsl.ActorTestKit
import org.scalatest.BeforeAndAfterAll

class AsyncForwardSpec
    extends AnyWordSpec
    with BeforeAndAfterAll
    with Matchers {

  val testKit = ActorTestKit()                                          ❶

  "a Simplified Manager" must {

    "actor gets forwarded message from manager" in {
      val manager = testKit.spawn(SimplifiedManager())                  ❷
      val probe = testKit.createTestProbe[String]()                     ❸
      manager ! SimplifiedManager.Forward("message-to-parse", probe.ref)
      probe.expectMessage("message-to-parse")                           ❹
    }
  }

  override def afterAll(): Unit = testKit.shutdownTestKit()             ❺
}
```

❶ Creates the test kit

❷ Spawns the actor under test

❸ Spawns the probe

❹ Tests the expectation

❺ Shuts down the ActorSystem

At the moment, this test is very similar to the sync examples, with one big difference: you are forced to shut down the `ActorSystem`.

Monitoring

Before you read the next section, you need to learn how to monitor an actor. Monitoring uses a probe to check whether a real actor receives a message. This is often necessary when you send a message to an actor that triggers a cascade of